

ALPS: Measuring Valid Creativity in Large Language Models with Mathematical Construction

Anonymous submission

Abstract

Large language models increasingly produce outputs presented as discoveries — new proofs, conjectures, molecules. Whether such an output that appears creative is truly original and effective is hard to establish: open-ended outputs require subjective judgment, the output may replicate something seen in training, or the task may be too simple to need creativity. We present ALPS (Austin-Law Proof-Synthesis), a benchmark that designs a task to measure *valid creativity*: producing a solution that is original and can be proven correct. Each instance is a single equational law, certified to require either the construction of an infinite mathematical structure satisfying the law, or a proof that no such structure exists. Submissions are verified by automated proof checking with no human involvement, and a public generator produces new instances without limit, so models are never evaluated on problems they may have seen. A portfolio of eight configurations of leading automated provers resolves 2.5% of the 4,141-law evaluation pool, and a twentyfold budget increase adds 0.1%: the obstacle is not compute, but the absence of any method that produces the tailored structure each law requires. Under a fixed protocol, the strongest reasoning model we test succeeds in 14% of instances on the proof side, but none on the construction side. ALPS is therefore released almost entirely unsolved, together with its corpus, generator, and automated judge.

1 Introduction

As large language models (LLMs) begin to take on scientific work in the real world - formalizing and proving theorems (Yang et al. 2023; Xin et al. 2024), proposing conjectures (Romera-Paredes et al. 2024; Gottweis et al. 2026; Lu et al. 2024), and designing molecules and materials (Jumper et al. 2021; Merchant et al. 2023) - AI-for-science has grown increasingly interested in measuring an LLM’s ability to reason through problems and produce creative solutions (Chen et al. 2025a). Recent results within mathematics have shown this interest to be well-founded. In 2026, a general-purpose reasoning model resolved the roughly eighty-year-old Erdős unit-distance problem, a long-open question in discrete geometry (OpenAI 2026). Producing a result like this requires

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

	Verifiable <i>no subjectivity</i>	Renewable <i>no contamination</i>	Constructive <i>no set procedure</i>
Static problem sets MATH, code, miniF2F	✓		✓
Synthetic puzzles random SAT, ETP	✓	✓	
Open discovery hypotheses, molecules		✓	✓
ALPS (ours)	✓	✓	✓

Table 1: Each property removes one confound in measuring valid creativity: verification removes subjective judging, renewal removes contamination, and construction removes solution by a general procedure. Each prior benchmark family (Hendrycks et al. 2021; Chen et al. 2021; Zheng, Han, and Polu 2021; Selman, Mitchell, and Levesque 1996; Bolan et al. 2025; Gottweis et al. 2026; Jumper et al. 2021) fulfills two of the three, but effectively measuring valid creativity requires all three.

creativity, but a solution that appears creative is not automatically a valid one. Two conditions must hold: creativity was necessary to solve the problem and the answer can be soundly justified. We call this combination valid creativity.

Current evaluation methods fail to reliably measure the validity of a creative output. For an evaluation to be effective, the solution must be *verifiable*, with an answer that can be checked objectively rather than judged. The problems must be *renewable*, generated fresh without limit so the model cannot have seen them in training. And the task must be *constructive*, posing problems that no general procedure solves, so the language model must make its own decisions rather than execute a routine. A benchmark admits a confound for each of the properties it lacks. Without renewability and constructiveness, creativity may not have been necessary: the problem could have been answered from memory or by a known general procedure. Every prior benchmark family has at most two of the three, depicted in Table 1. Synthetic puzzles such as random SAT (Selman, Mitchell, and Levesque 1996) and the Equational Theories Project (Bolan et al. 2025) are verifiable and renewable, but a general procedure solves their problems given enough compute, so success can mea-

sure the ability to execute a known method rather than to create. Static corpora such as MATH (Hendrycks et al. 2021), coding benchmarks (Chen et al. 2021), and miniF2F (Zheng, Han, and Polu 2021) ask for constructed, checkable solutions, but draw from fixed sets that eventually contaminate training data, so a high score may reflect memorization (Xu et al. 2024). Open scientific discovery, such as hypothesis generation (Gottweis et al. 2026; Lu et al. 2024) and molecule design (Jumper et al. 2021), demands construction and never exhausts its supply, but its answers are judged subjectively, so scores depend on the judge.

Formal mathematics is the one setting where each of these three properties can be fulfilled simultaneously. A formal proof can be checked by machine, and fresh problems can be generated without limit, so verifiability and renewability come immediately. Construction is the property that must be established by design. Equational theory (Birkhoff 1935) studies *magmas*: sets with a single binary operation $\diamond$ (Burris, Sankappanavar et al. 1981). Its *laws* are universally quantified equations that an operation may or may not obey, such as commutativity $x \diamond y = y \diamond x$ (Burris, Sankappanavar et al. 1981; Baader and Nipkow 1998). A magma is a *model* of a law when the equation holds under every assignment of its elements (Burris, Sankappanavar et al. 1981). The one-element magma has a single element, so the identity $x = y$ holds, meaning every element equals every other and any equation is then true. This is named the *trivial* magma (Burris, Sankappanavar et al. 1981), as it models every law and carries no information. What carries content is whether a law has a nontrivial model, one with at least two elements. Within a finite set, such a model can always be found by search, since only finitely many operations exist to test. Some laws, however, have no nontrivial finite model at all. A law that nonetheless has a nontrivial model, necessarily an infinite one, is called an *Austin law* (Kisielewicz 1997; Bolan et al. 2025), and no finite search can reach its model. For an Austin law, a model cannot be enumerated; it must be constructed and then justified.

We introduce the Austin-Law Proof-Synthesis (ALPS) benchmark. Each instance is a law with no nontrivial finite model, a property established in advance, which leaves exactly two possibilities: the law is trivial, collapsing every model so that $x = y$, or it is Austin, with an infinite model that satisfies it. Solving an instance means deciding which holds and proving it: a deductive proof on the trivial side, a constructed infinite model on the Austin side. Every answer is settled by an automatically validated proof certificate, so grading is objective and needs no human judge. The instances come from a generator that proposes candidate laws without limit, then screens each to keep only those with this property, so ALPS never runs short of fresh, unseen problems.

Austin laws sit where AI-for-science and mathematical provers are converging. The recent Erdős unit-distance result was settled by a reasoning model through the construction of an infinite object, at the frontier of what current language models can do (OpenAI 2026). Computational mathematical provers approach this same frontier from the other side with new methods of constructing infinite models (Janota, Rawson, and Schulz 2026). Austin laws are this challenge

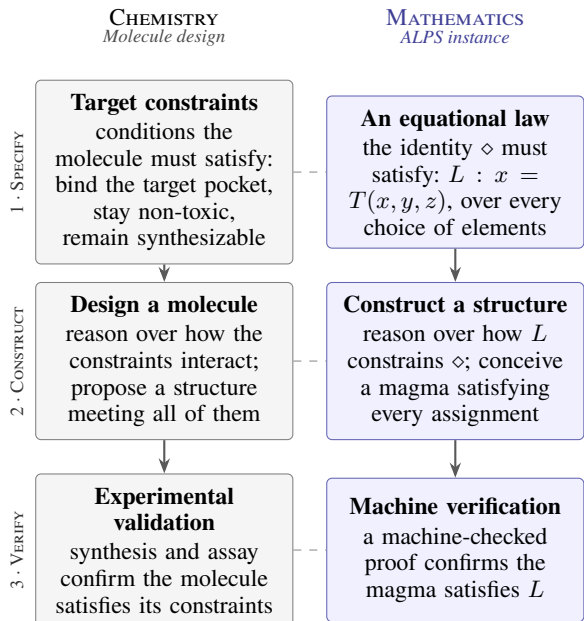


Figure 1: Scientific discovery proceeds in three phases regardless of domain — specify, construct, verify — instantiated here by molecule design (left) and an ALPS instance (right). A specification fixes what an acceptable answer must satisfy, a candidate structure is constructed to meet all requirements, and an independent check verifies whether it does: an experiment for the molecule, a proof certificate for ALPS.

in renewable form: each is an infinite-construction problem with a definite answer, and fresh ones can be generated without limit.

What ALPS requires of the solver reaches well beyond equational theory. Resolving a law means proposing a new mathematical structure under constraints and establishing its global properties: the law fixes conditions the operation must satisfy on every combination of elements, the solver chooses a carrier and defines an operation intended to meet them, and a formal proof settles whether it does, across the infinitely many cases the solver can never check directly. Constructing a mathematical object that provably meets such a specification is itself an act of scientific discovery. The process by which this is executed - specification, construction, and verification - recurs across the sciences (Figure 1). For example, in molecule design, the specification is a target profile and the check an experiment; in ALPS, a law and a formal proof. Because that proof decides both conditions of valid creativity from the answer alone, a verified ALPS solve is direct evidence that the model created a novel structure and justified it. As a result, ALPS directly measures the capability discovery demands, in a form that can be generated and checked without limit.

We demonstrate empirically that the task ALPS defines is constructive. A portfolio built from eight configurations of the strongest automated provers (Kovács and Voronkov 2013; Schulz, Cruanes, and Vukmirović 2019; Smallbone 2021),

resolves 2.5% of the 4,141 laws that survive screening, and a twentyfold increase in per-prover budget adds 0.1%. The remaining laws yield neither a model nor a triviality proof under any configuration we test, although each is certified to have a determinate answer; they are limited by the methods available rather than by the compute allotted to them. In full, our contributions are:

- a two-sided construction task on Austin laws, with every instance certified to have a determinate answer;
- a corpus of such laws and a public generator that mints fresh instances without limit;
- an automated judge that certifies submissions through independent channels - a Lean proof (Moura and Ullrich 2021) for triviality, and an automated-prover certificate (Kovács and Voronkov 2013) for constructed models - with no human in the loop; and
- a strong automated baseline that establishes the method-bound hard tier and, along the way, settles an order-5 case left open by the Equational Theories Project (ETP) (Bolan et al. 2025).

2 Related Works

2.1 AI-for-Science Benchmarks

Existing benchmark families each hold two of the three properties ALPS targets and fall short on the third. Static problem sets are fixed collections spanning grade-school and competition mathematics (Cobbe et al. 2021; Hendrycks et al. 2021), graduate science (Rein et al. 2023), coding (Chen et al. 2021), and formal proof construction (Zheng, Han, and Polu 2021; Azerbayev et al. 2023; Tsoukalas et al. 2024). These are scored objectively and require reasoning through each problem rather than recalling a memorized algorithm. However, because their problems are fixed, these benchmarks inevitably suffer from contamination, as their evaluation set will eventually enter the training data of the language models they are meant to test, rendering further evaluation on this benchmark unreliable (Xu et al. 2024; Chen et al. 2025b). Some benchmarks attempt to delay contamination by holding problems secret (Glazer et al. 2024) or releasing new problems over time (White et al. 2025; Jain et al. 2025), but will always be limited by the rate at which humans can author new problems.

Synthetic puzzles avoid this limit by generating problems from a fixed pattern or rule so that fresh instances can be produced without end, such as random SAT formulas (Selman, Mitchell, and Levesque 1996), graph-generated reasoning tasks (Zhu et al. 2024), Sudoku puzzles (Seely et al. 2025), or the equational laws of the Equational Theories Project (Bolan et al. 2025). However, the same regularity that makes these problems generable also makes them solvable by a known procedure. Therefore, these benchmarks often test the LLM’s ability to execute a known method rather than to construct a creative solution.

Open scientific discovery benchmarks evaluate an LLM’s ability to construct novel, useful scientific advancements such as hypotheses, conjectures, designs, and mathematical structures (Lu et al. 2024; Gottweis et al. 2026; Alkan

et al. 2025; Romera-Paredes et al. 2024; Merchant et al. 2023; Jumper et al. 2021). These pose a challenging task, requiring the LLM to invent a new solution rather than retrieve or execute a known procedure. However, their results are validated by human expertise or physical experiment, making evaluation subjective and slow. ALPS uniquely combines the strengths of each benchmark family: objective scoring, endless renewal, and a constructive task that no known general procedure solves.

2.2 Theorem Proving and Verification

ALPS scores answers using machine verification, without a human in the loop, made possible by theorem provers. Lean (Moura and Ullrich 2021) is a functional programming language equipped with a proof assistant whose kernel can automatically check the validity of proofs, such that a certificate is guaranteed to be correct. Each proposition, including the statement that a magma satisfies a law, is encoded as a type within the language, and a proof of it is a term inhabiting that type, exactly as a value inhabits a type in an ordinary programming language. Checking a proof is resolved using type checking: the kernel confirms that submitted terms have the claimed type, similar to a compiler checking that a function’s body matches its signature. A statement quantified over infinitely many inputs, such as a law holding under every assignment, becomes a function type. Within Lean, its proof is a function returning a proof for an arbitrary argument, and the kernel type-checks that function once, symbolically. A certificate that type-checks successfully is therefore correct by construction and automatically validated.

Unlike checking them, finding proofs is difficult. Many works investigate automating this search. Classical automated theorem provers such as Vampire and E (Kovács and Voronkov 2013; Schulz, Cruanes, and Vukmirović 2019) search for first-order proofs by superposition calculus (Bachmair and Ganzinger 1994), a strategy that repeatedly derives logical consequences of the input until it reaches a contradiction or no new consequences remain. More recently, language models have been trained to construct proofs in systems like Lean (Yang et al. 2023; Xin et al. 2024; Hubert et al. 2026), measured on formal proof benchmarks (Zheng, Han, and Polu 2021).

2.3 Equational Reasoning

Equational theory studies the foundation of algebra - the identities that operations satisfy and the entailments among them. The Equational Theories Project (Bolan et al. 2025) studies this at scale, enumerating the implications among thousands of small laws and settling them automatically using a pipeline of automated provers and finite-model finders. This work singles out the laws whose only nontrivial models are infinite, the Austin laws (Kisielewicz 1997), as the hardest to resolve, since neither a searchable finite model nor a short derivation settles them.

Several automated methods are tailored to equational laws. Paradox and Mace4 (Claessen and Sörensson 2003; McCune 2003) are designed to find finite models when they exist, while Infinox (Claessen and Lillieström 2011) instead proves that no finite model exists. Twee (Smallbone 2021)

implements unifying Knuth-Bendix completion (Knuth and Bendix 1970; Bachmair, Dershowitz, and Plaisted 1989), orienting equations into rewrite systems to solve equational problems. When a saturation from Twee or a superposition prover halts without reaching a contradiction, the clauses it has accumulated already describe a model. Janota, Rawson, and Schulz (2026) recently studied this for equational theories, extracting an explicit model from the saturation in a case study.

3 The Austin-Law Proof-Synthesis Benchmark

ALPS is built in three parts, displayed in Figures 2 and 3: a two-sided task posed over laws certified to have no nontrivial finite model, a corpus construction that mints such laws at any order by extending known Austin laws and screening the results, and an automated judge that verifies answers to either side of the task with a machine-checked certificate. We describe each component below. Each part describes one column of Table 1: the admissibility certificate makes every instance a construction problem with a determinate answer, the extension engine renews the supply without limit, and the judge verifies answers with no human in the loop.

3.1 Problem Formulation

A magma is a set M , called the carrier, together with a single binary operation $\diamond: M \times M \rightarrow M$ and no further axioms. An equational law is a universally quantified identity between two terms built from variables and $\diamond$. Its order is the number of times the operation appears, so the commutative law $x \diamond y = y \diamond x$ has order two. A magma *satisfies* a law when the identity holds under every assignment of carrier elements to its variables, and is then called a *model* of the law.

Every ALPS instance is a law of the form $L: x = T[x, y, z, \dots]$: a designated variable x , isolated on the left-hand side, equated with a term T of order five or more, where the brackets list the variables from which T is built; x may itself recur within T . The trivial magma contains only one element, and is thus a model of every law, carrying no information. We call a model nontrivial when its carrier holds at least two distinct elements. The question ALPS poses for each L is whether a nontrivial model exists at all: either L entails $x = y$, collapsing every model to a single value, or some nontrivial magma satisfies it.

A nontrivial model, when one exists, may be finite or infinite. The finite case is a matter of search: enumerating operations on carriers of increasing size must eventually find it, requiring none of the law-specific creativity the benchmark aims to measure. We therefore admit a law into the benchmark only when it carries a machine-checked proof that no nontrivial finite model exists, calling such laws *admissible*. Admissibility forces a dichotomy: every law either entails $x = y$ or it does not, so each instance has a determinate answer. Since laws are universally quantified, $L \models x = y$ states that any two elements of any model are equal. For an admissible law, the two cases are exactly

$$\underbrace{L \models x = y}_{\text{trivial}} \quad \text{or} \quad \underbrace{L \text{ has an infinite nontrivial model}}_{\text{Austin}}. \quad (1)$$

Laws of the second kind are the Austin laws (Kisielewicz 1997).

The asymmetry between the two sides of the task is essential to what the benchmark measures. Whether L entails $x = y$ is a question of first-order logic, the language of statements built from the operation $\diamond$, equality, logical connectives, and quantifiers ranging over carrier elements (Baader and Nipkow 1998). Proof search within first-order logic is complete: if the entailment holds, a systematic search will eventually confirm it. The trivial side is, in principle, settled by deduction. The Austin case admits no such guarantee. A search that fails to derive $x = y$, however long it runs, yields no model. Because equational entailment is undecidable in general (Baader and Nipkow 1998), no uniform procedure can be expected to close the gap. Establishing that a law is Austin means inventing an infinite magma tailored to the individual law to satisfy it. This synthesis, which deductive search does not supply, is the capability that ALPS isolates.

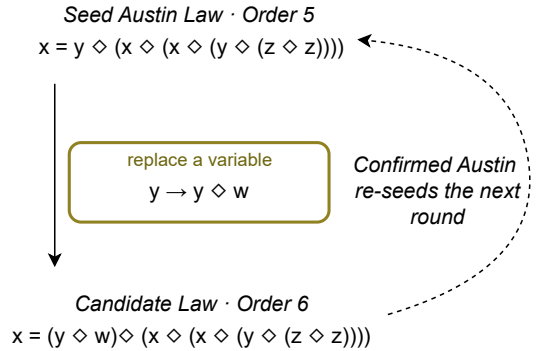


Figure 2: Seed Austin laws are extended by replacing one variable occurrence v with $v \diamond w$ or $w \diamond v$. Laws confirmed Austin re-seed the next round, so generation applies recursively at every order.

3.2 Corpus Construction

Generation by extension. Admissible laws are too rare for random sampling to produce in quantity, and the few it does are often redundant. We instead grow the corpus outward from laws already known to be Austin (Figure 2): a seed law $x = T$ is extended by selecting any variable v in T and replacing one occurrence with $v \diamond w$ or $w \diamond v$ for some other variable w , yielding higher-order candidates. Because a candidate inherits most of the seed’s term structure, it often inherits the mechanism that prevents a finite model, increasing the likelihood of finding new admissible laws relative to random sampling. The construction is recursive - laws confirmed Austin at order n seed the round at order $n + 1$ - so the corpus has no terminal order. This enables ALPS to be

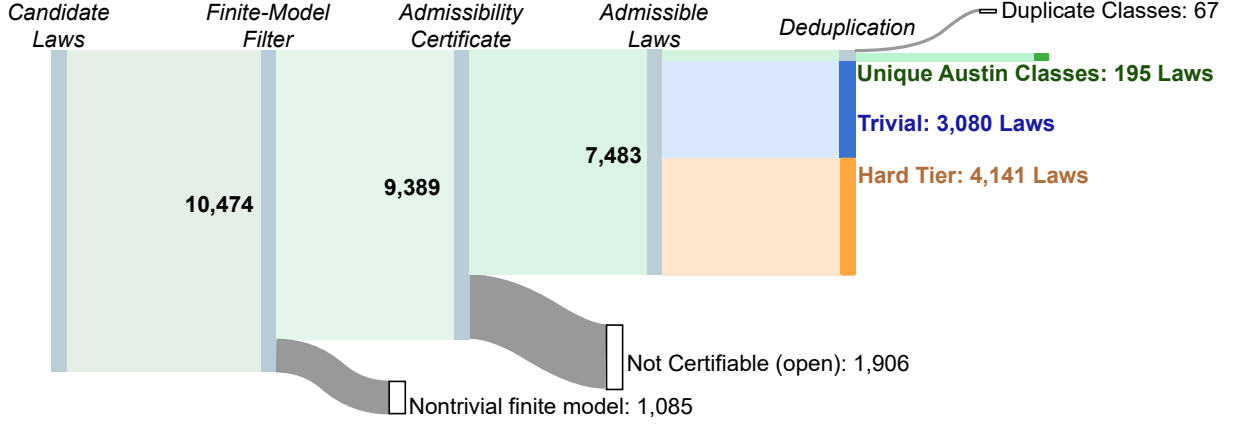


Figure 3: Each generated candidate passes several screens to guarantee quality. A finite-model filter discards laws with small nontrivial models. The admissibility prover certifies the rest have no finite model and discards any that it cannot certify, labeled “(open)” to signify that some discarded candidates may be admissible. Proofs generated throughout the screening process help classify the admissible law pool into trivial, Austin, or the unresolved hard tier.

renewable: fresh instances can be minted after any training cutoff, so evaluation never depends on problems a model may have seen.

Admissibility. Each candidate then enters the screening pipeline shown in Figure 3. A finite-model filter searches carriers of increasing size, discarding every candidate it can satisfy nontrivially. The survivors face the admissibility prover, which attempts a machine-checked proof that no nontrivial finite model exists. The claim cannot be posed to the prover directly because finiteness itself is not expressible in first-order logic. However, on a finite carrier, an injective map must also be surjective. Unlike finiteness, surjectivity is a first-order statement. The argument therefore has two steps - injectivity of a designated subterm of T , which follows from the law itself, and surjectivity of that subterm, which is asserted and holds in every finite model.

Let S be a subterm of T that contains every occurrence of x , so that the law factors as

$$x = C[S(x, \bar{y})], \quad (2)$$

where C is the part of T surrounding S , C does not contain x , and $\bar{y}$ abbreviates the law’s remaining variables. We read S as a map sending a carrier element a to $S(a, \bar{y})$, with $\bar{y}$ held fixed. Injectivity of S is the claim that no two elements share an image: if $S(a, \bar{y}) = S(b, \bar{y})$, then $a = b$. Equation (2) supplies this directly. It states that applying C to $S(x, \bar{y})$ recovers x - meaning C is a left inverse of S - so applying C to both sides of $S(a, \bar{y}) = S(b, \bar{y})$ gives

$$a = C[S(a, \bar{y})] = C[S(b, \bar{y})] = b.$$

Thus, $S(a, \bar{y}) = S(b, \bar{y})$ implies $a = b$. The derivation uses only the law itself, so S is injective in every model of L , and no proof search is required. For the second step, because S is injective in every model, it must be surjective in every finite model, meaning every element u arises as $S(x, \bar{y})$ for some x : each finite model of L satisfies the first-order statement

$$\text{surj}(S): \quad \forall \bar{y} \forall u \exists x: S(x, \bar{y}) = u. \quad (3)$$

The admissibility query submits both steps to the prover as a single derivation task, notated as $\vdash$ in contrast to the entailment $\models$ of Equation (1):

$$\{L, \text{surj}(S)\} \vdash x = y. \quad (4)$$

The final admissibility certificate is thus a refutation proof of Equation (4), which establishes that $x = y$ is derivable from L together with $\text{surj}(S)$. Since a derived statement holds in every model of its premises, any model that satisfies both L and $\text{surj}(S)$ also satisfies $x = y$. Every finite model of L satisfies both - L as its model, $\text{surj}(S)$ by the argument above - so every finite model of L is certified to be trivial. This leaves only the two cases of Equation (1): either L entails $x = y$ and every model collapses, or L has a nontrivial model, which must then be infinite. Admission requires this certificate, and none exists for a law with a nontrivial finite model. The cost is incompleteness: when no subterm of T contains every occurrence of x and injectivity cannot be proven otherwise, nothing is certified. Such candidates are discarded, even though some may be admissible. The screening provers classify the admissible pool into laws proven trivial, laws proven Austin, and the residual that the baseline of the next section attempts.

Deduplication. Because every law descends from a small seed set, an extension may pose the same problem as its seed or as a sibling. Two laws pose the same problem when each entails the other:

$$L_1 \models L_2 \quad \text{and} \quad L_2 \models L_1, \quad (5)$$

in which case the two laws have the same models and are called equivalent. A pair is distinct when either direction of Equation (5) fails, and a failure is witnessed by a separating model: a magma that is a model of L_1 but not of L_2 shows

A toy inadmissible law $L_0 : x = x \diamond (y \diamond y)$

Submitted presentation $E_0 : u \diamond v = u$
the operation returns its left argument

The two checks

(a) E_0 entails L_0 . Rewriting the right-hand side with $u \diamond v = u$ ($u = x, v = y \diamond y$):

$$x \diamond (y \diamond y) \longrightarrow x, \text{ so both sides of } L_0 \text{ agree.} \quad \checkmark$$

(b) $E_0 \cup \{a \neq b\}$ is satisfiable. The two-element magma

$\diamond$	a	b
a	a	a
b	b	b

has two distinct elements and satisfies $u \diamond v = u$ in every entry, so it is a model of E_0 with $a \neq b$; E_0 does not force $a = b$. $\checkmark$

Conclusion E_0 has a two-element model that satisfies L_0 , so L_0 has a nontrivial model.

Figure 4: The construction certificate on a toy, inadmissible law. The solver submits equations E , not a model; the judge accepts when a prover certifies that E entails the law (check a) and has a model with two distinct elements (check b). Here that model is finite, which is why L_0 is inadmissible; for an admissible law the same two checks apply and admissibility forces the model to be infinite.

$L_1 \not\models L_2$. The models constructed during screening double as these witnesses at no additional prover cost. Across the 34,191 pairs among the 262 laws proved Austin, 33,936 pairs are separated by a model already found during screening. The prover settles the remaining pairs by deriving both directions of Equation (5): 255 pairs are proved equivalent, and no pair is left undecided. Equivalence is transitive: if L_1 and L_2 are equivalent and L_2 and L_3 are equivalent, then every model of L_1 is a model of L_2 , and additionally of L_3 , so L_1 and L_3 are equivalent as well. The 255 proved equivalences therefore chain into groups of mutually equivalent laws, and each group counts as a single problem, represented by one law. A group of k laws accounts for $k(k-1)/2$ of the proved pairs but only $k-1$ duplicates, which is why 255 pairs merge away only 67 laws: the 262 laws collapse to 195 classes. We report corpus size in classes rather than laws, so the count reflects only distinct problems and no solution is credited twice.

3.3 Answer Verification

Each solved ALPS instance is a machine-verified mathematical statement. The two sides of the task require different verification processes, so the judge issues a certificate through the appropriate channel: a proof checked by the Lean kernel for the trivial side (Moura and Ullrich 2021), and an automated-prover certificate for the constructed model (Kovács and Voronkov 2013). Both channels are fully automatic, and a submission is accepted only if its check succeeds. Verifica-

tion rests on two trusted checkers, the Lean kernel on the trivial side and prover saturation on the construction side, cross-checkable by a second prover (Smallbone 2021).

The trivial channel. For each law, the judge generates the proposition

$$\text{TRIVIALGOAL} \equiv \forall M \forall \diamond : M \times M \rightarrow M, \quad \text{Law}(\diamond) \rightarrow \forall a, b : M, a = b, \quad (6)$$

where $\text{Law}(\diamond)$ asserts that $\diamond$ satisfies L . In words, Equation (6) states that any magma satisfying L has all of its elements equal: it is the trivial side of Equation (1), with the magma quantified inside the statement so that a single proof covers every carrier and every operation at once. A submission is a Lean proof of Equation (6). The statement is generated by the judge, never written by the submitter: the judged file fixes Equation (6) in a header, inserts the submitted proof, and requires it to elaborate at exactly that type, and the proof’s axiom footprint must fall within a fixed allowlist, which blocks declared axioms and placeholder proofs that would allow a solver to postulate what it must prove. Under these constraints, a proof the kernel accepts is correct by construction (Moura and Ullrich 2021). In practice, such a proof formalizes a chain of equalities $a = t_1 = \dots = t_k = b$ in which each step applies one instance of L .

The construction channel. A model of an Austin law has an infinite carrier, so the model itself cannot be submitted: its operation table is infinite, and a submission must be a finite object a machine can check. The solver instead submits a finite presentation E : a finite set of equations over $\diamond$, whose models are the magmas that satisfy every equation in E . E must satisfy two claims: (i) every model of E satisfies L , and (ii) some model of E has two distinct elements. The claims constrain E from opposite sides: it must be strong enough to entail the law, yet weak enough that its models do not all collapse to a single element. The judge accepts only when a prover certifies both. Figure 4 demonstrates this process on a toy law. Formally, the two queries are

$$(a) E \vdash L \quad (b) E \cup \{a \neq b\} \text{ is satisfiable.} \quad (7)$$

Check (a) is a derivation task of the same kind as Equation (4): the prover returns a refutation proof that L is derivable from the equations of E , and by the same reasoning as before, every model of E satisfies L , which is claim (i). Check (b) asks whether some magma satisfies every equation of E while keeping two elements a and b distinct. The prover answers by saturation: starting from $E \cup \{a \neq b\}$, it derives consequences until either a contradiction appears or nothing new can be derived, at which point the set is called saturated. A saturated set that contains no contradiction is satisfiable, provided no inference was skipped along the way, and its accumulated consequences describe a model (Bachmair and Ganzinger 1994). Provers do skip inferences in some configurations, trading completeness for speed, and report when they have done so. The judge therefore accepts check (b) only when the prover reports that its strategy retained completeness. A terminating, complete saturation of $E \cup \{a \neq b\}$ thus establishes claim (ii). The two checks compose: by (ii) some

magma satisfies E with $a \neq b$, by (i) that magma satisfies L , so L has a nontrivial model, and the admissibility certificate of the previous section forces that model to be infinite. L is therefore Austin. Neither query mentions infinity: both are finite first-order tasks, and infinitude enters only through the admissibility certificate the instance already carries.

Although grading is fully automatic, the checks themselves reject any submission that does not encode a genuine construction to ensure a certificate cannot be earned without solving the instance. Submitting the law itself as E gains nothing: check (a) becomes immediate, but check (b) reduces to the saturation of $L \cup \{a \neq b\}$, which is the computation that fails to terminate on the very laws the solver is being asked about. A collapsing presentation, such as $\{x \diamond y = x, x \diamond y = y\}$, entails every law, because its two equations give $a = a \diamond b = b$ for any two elements — but for the same reason it forces $a = b$, so $E \cup \{a \neq b\}$ is unsatisfiable and check (b) fails. A presentation too weak to force the law, such as the left projection $\{x \diamond y = x\}$, passes check (b) but fails check (a). The judge is validated against controls of all three kinds before any evaluation run: a known-correct presentation certifies, the collapsing presentation is rejected at check (b), and the left projection is rejected at check (a). The submission format additionally admits only unconditional equations, so implications, disequations, and literal restatements of $x = y$ never reach the prover.

A single Lean judge covering both sides is not currently possible. The model families that are straightforward to formalize in Lean are arithmetic: operations given by polynomial or affine formulas over $\mathbb{Z}$, $\mathbb{Z}^k$, or $\mathbb{Z}/n$. No model of an admissible law is among them, since an arithmetic operation satisfying the law would yield a nontrivial finite model, contradicting admissibility. The models that do exist are term models built from saturations (Janota, Rawson, and Schulz 2026), and formalizing these in Lean awaits a proof of ground confluence. The construction side is therefore graded by the prover certificate.

4 Experiments

4.1 The Automated Frontier

We establish what automation resolves using a portfolio spanning eight configurations of the leading off-the-shelf provers available: Vampire 5.0.1 in five modes (proof search and saturation, under Knuth-Bendix and lexicographic path orderings) (Kovács and Voronkov 2013), the E prover in a proof and a saturation mode (Schulz, Cruanes, and Vukmirović 2019), and Twee 2.6.1, an implementation of unfailing completion built for equational reasoning (Smallbone 2021), each running over per-prover budgets from 30 to 600 seconds. A law counts as resolved when any configuration returns a certificate. Finite-model finders such as Mace4 (McCune 2003) and Paradox (Claessen and Sörensson 2003) cannot succeed and are omitted, as our admissibility classification certifies that no nontrivial finite model exists. Saturation and completion are automated methods in the portfolio that can exhibit a model: a saturation that terminates without contradiction describes one (Section 3.3).

Sweeping the portfolio over the admissible pool produces

Budget (s)	New models	New trivial	Unresolved
30	4	69	4,068
60	0	7	4,061
120	0	4	4,057
300	0	3	4,054
600	0	3	4,051

Table 2: Newly resolved laws at each per-configuration time budget, from the sweep of the 4,141-law admissible residual. All four models are constructed at the 30-second budget; no model is constructed at any longer budget. New triviality proofs fall from 69 to 3, and 4,051 laws (97.8%) remain unresolved at 600 seconds.

two populations by resolution time. The models of the 262 laws proved Austin during screening are constructed in a median of 0.1 seconds, and sweeping the 4,141-law residual with the full portfolio reclassifies a further 90 laws (2.2% of the residual): 86 triviality proofs and 4 completion-only models. The remaining 4,051 laws resolve under no configuration at any budget up to 600 seconds and constitute the hard tier. The two populations are separated by a gap in the resolution-time distribution rather than by a threshold we impose.

If the unresolved laws were categorized into the hard tier due to insufficient compute, additional budget would be expected to continue resolving new laws at a comparable rate, including new models. Instead, of the 90 residual laws the sweep resolves, 73 resolve at the 30-second per-configuration budget. Raising the budget by a factor of 20 resolves only 17 more, each one a triviality proof (Table 2). All four models the sweep recovers, together with the 262 constructed during screening, are identified within the first 30 seconds. Added compute continues to prove residual laws trivial at a declining rate, but constructs no model the shortest budget did not already reach.

The resolved tier contains a result of independent interest. Among the laws the portfolio resolves is an order-five law and its dual (12857 and 33436 in the ETP numbering) that the Equational Theories Project recorded as having only trivial finite models, leaving the existence of an infinite model open (Bolan et al. 2025). Two independent methods, a Vampire saturation and a Twee completion, terminate on the pair without deriving $x = y$. As established in Section 3.3, a terminating, complete run describes a model, here one in which $x = y$ fails, and admissibility forces that model to be infinite. We additionally verify the extracted presentation, a set of 27 equations, directly: it satisfies the law and does not force $x = y$. The model is of the saturation-derived kind described in Section 3.3, so its Lean formalization awaits the ground-confluence proof noted there; the full worked example is in the appendix.

4.2 The LLM Frontier

Language models enter ALPS on the same terms as the automated portfolio: nothing in the task references how an answer is produced, only that it certifies. We evaluate them under a

Model	Certified-easy ($n = 63$)			Hard ($n = 25$)
	full support	no waypoints	low effort	construction
GPT-4.1	0	0*	—	0
o4-mini	0	0*	0*	0
o3	9 [†]	6	0*	0

Table 3: Verified LLM solves under the fixed protocol (identical inputs, three feedback rounds against the judge). Full support = waypoint lemmas in the prompt, medium reasoning effort; trivial-side solves are Lean-verified, construction submissions are certified by the automated prover. The no-waypoints column for o3 reruns the full 63-law set. *Settings not rerun in full: they cover only the laws solved at full support, and since waypoints and reasoning effort can only help, laws unsolved at the stronger setting are counted as unsolved. [†]Pass@1 from a single run; an identical rerun reproduces one of the nine solves and adds one new law (see text).

fixed protocol designed so that failures are attributable. Every model receives identical inputs — the law, the task statement, and a fixed answer format — and may submit to the judge up to three times, receiving each verdict as feedback. On the trivial side, the formalization burden is removed. The model emits a chain of terms, each step being one application of the law. The autoformalization harness matches each step, bridging short gaps and assembling the Lean proof itself, so the test is finding the derivation rather than writing Lean. We test varying levels of support. With no support, the prompt contains only the law; at full support, it also includes waypoint lemmas: intermediate consequences of the law, derived by a prover, through which a collapse chain can pass. The harness is frozen across models. Saturation provers are withheld from the models, since with them, a language model can re-run the portfolio at a larger budget. The judge and the harness are the tools available to any solver built on ALPS.

To evaluate the two sides of the task, we construct one law set for each. The certified-easy set tests whether a model can produce a verified solution under the most favorable conditions the corpus offers: 63 laws from the trivial tier whose collapse chains — derivations $a = t_1 = \dots = t_k = b$ of the kind the harness assembles — are short and simple. Rewriting with the law from both ends until the two chains meet resolves each of the 63 in under five seconds, and most chains are only three law applications long. The machine-found solutions are withheld from release, so a solver cannot reproduce a published answer.

The construction experiment is posed over the hard tier itself: we sample 25 of the 4,051 laws that no automated configuration resolves. These laws are open in both directions. Each is admissible, so it either entails $x = y$ or has an infinite nontrivial model, and no method has determined which; a construction can certify only in the second case. Because which side holds is unknown, we evaluate both. Each law is attempted through the construction channel, in which the model proposes a presentation E and the prover certifies entailment and nontriviality (Section 3.3), and through the trivial channel with no support, since waypoint lemmas do not exist for unresolved laws. A solver can therefore achieve a verified solve on whichever side it can certify, and a certified answer in either direction would settle an open law.

Table 3 reports the trivial-side results. At full support, o3 at

medium reasoning effort produces nine verified solutions on the 63 certified-easy laws (14% pass@1). GPT-4.1 and o4-mini produce none on the same laws under the same protocol. The verified chains are three to four law applications long, and most are accepted on the second or third feedback round, at a median of roughly 50,000 completion tokens per solved law. Failures share one dominant pattern at every setting: the submitted chains contain steps that are not applications of the law — no substitution instance of either side of L rewrites one term into the next — and the harness rejects the chain at the matching stage. Setting o3’s reasoning effort to low reduces the number of solves from nine to zero. Verified solves therefore occur only for the strongest reasoning model we test, and only above the lowest effort setting.

Solve events are stochastic. A second run under identical settings reproduces one of the nine solves and newly solves one of five matched laws the first run missed; across the two runs, ten distinct laws have verified solutions. Single-run counts should therefore be read as pass@1 samples rather than as a fixed solved set: which laws certify varies between runs, and the union of verified laws grows with repetition. The signals that are stable across every run are binary ones: o3 produces verified solves in each run, GPT-4.1 and o4-mini produce none in any, and low reasoning effort produces none in any. This variation is expected under the protocol: each run samples a fresh reasoning trace from a general-purpose model under a fixed prompt, and the harness adds no search, ensembling, or retry beyond the three feedback rounds, so marginal solves appear and disappear across runs. These results are therefore a lower bound on language-model performance on the task.

The support ablation withholds the waypoint lemmas and reruns the full certified-easy set: without support, o3 produces six verified solves on the 63 laws. Three of the nine laws solved at full support certify again, and three laws that full support missed certifies for the first time, bringing the union of verified laws across all runs to 13. These certificates establish that waypoints are not required for a verified solve; the model finds some short derivations unaided. The size of the support effect, however, is not identifiable from these runs. Under identical full-support settings, a second run reproduces only one of the nine solves, so the difference between six and nine is within run-to-run variation, and the

A hard-tier law

$$L : x = y \diamond (x \diamond ((w \diamond y) \diamond ((y \diamond z) \diamond (y \diamond x))) \diamond y))$$

o4-mini’s submission

$$E_1 : y \diamond (x \diamond y) = x \quad E_2 : x \diamond (T \diamond y) = x \diamond y$$

T can represent any term

The two checks

(a) $E \vdash L$: rewriting L using T for the law’s middle term,
 $y \diamond (x \diamond (T \diamond y)) \stackrel{E_2}{=} y \diamond (x \diamond y) \stackrel{E_1}{=} x \quad \checkmark$

(b) $E \cup \{a \neq b\}$ *unsatisfiable*: $T \stackrel{E_1}{=} y \diamond (T \diamond y) \stackrel{E_2}{=} y \diamond y$
 — every element equals $y \diamond y$, so $a \neq b$ cannot be
 demonstrated under this E_1 and $E_2 \quad \times$

Rejected: The proposed equations do not prove $L \rightarrow x = y$

Figure 5: A rejected submission on the hard tier from the o4-mini language model. The proposed equations entail the law only when all elements are equivalent.

appearance of a new solve under reduced support is consistent with the same variation. We treat the support comparison as suggestive rather than as a measured effect.

None of the construction attempts from the LLMs succeed. o3, o4-mini, and GPT-4.1 each attempt the 25 hard-tier laws over three feedback rounds, and no submission passes both checks. On the same laws, the trivial channel with no support also produces no verified solution, so every sampled law remains open in both directions. Classifying each law by its final-round submission, 23 of o3’s 25 presentations are overly strong, entailing the law but collapsing: check (a) passes, and check (b) reports $E \cup \{a \neq b\}$ unsatisfiable, so the proposed structure admits only one-element models. The remaining two are too weak, maintaining two distinct elements but failing to entail the law. o4-mini divides into 11 that are too strong, 4 that are too weak, and 10 that fail both checks; GPT-4.1 divides into 10, 11, and 4. Figure 5 shows one such submission. On final submissions, o3 clears exactly one check on all 25 laws, GPT-4.1 on 21, and o4-mini on 15. Although every such submission scores as a failure due to the certificate requiring both checks to pass, the failure profile distinguishes the LLM performances. o3 never fails both checks, and its failures concentrate in a single mode. The weaker LLMs spread across all three modes, failing both checks on 10 (o4-mini) and 4 (GPT-4.1) of the 25 laws. A presentation that is neither too strong nor too weak is the capability the construction side isolates.

ALPS therefore launches with nearly all of its content yet to be solved. The automated portfolio resolves 2.2% of the admissible pool, the strongest language model we test solves 14% of the certified-easy set at pass@1, a set covering 63 of the 3,080 trivial-tier laws, and no LLM-based system we run produces a certified construction. Progress is measurable at every level above this floor: solves beyond the certified-easy set, higher per-law solve probabilities under repeated runs, and a first certified construction, which would settle an

open law. The evaluation sets contain only laws minted by the extension procedure, and the generator can generate new laws indefinitely beyond the released corpus, so future evaluations can draw instances created after any model’s training cutoff.

5 Conclusion

We presented ALPS (Austin-Law Proof-Synthesis), a benchmark that measures valid creativity in language models: the capability of producing a solution that is original and provably correct. Each instance is an equational law certified to have no nontrivial finite model, so the law either entails $x = y$ or is satisfied by an infinite magma, and a solution to either side is a validated proof certificate. The design secures both conditions of valid creativity through its three properties. Because answers are verifiable, acceptance is decided entirely by the certificate: a solution is graded correct only when its proof validates, independent of any human judgment. The corpus is renewable through its generator, which mints fresh instances past any training cutoff and screens each for admissibility automatically, making ALPS contamination-free. The task is demonstrated to be constructive empirically: a portfolio of eight configurations of leading automated provers resolves 2.2% of the 4,141-law evaluation pool, a twentyfold budget increase adds 17 triviality proofs and no models, and the strongest reasoning model we test certifies 14% of the certified-easy set on the trivial side and no construction on any law. Performance nonetheless improves with increased reasoning capacity, with the strongest model’s submissions satisfying one of two certification checks on every law, whereas weaker models sometimes fail both. ALPS launches nearly unsolved; we release its corpus, generator, and judge, and a first certified construction would settle a law that no existing method resolves.

References

- Alkan, A. K.; Sourav, S.; Jablonska, M.; Astarita, S.; Chakrabarty, R.; Garuda, N.; Khetarpal, P.; Pióro, M.; Tanoglidis, D.; Iyer, K. G.; et al. 2025. A survey on hypothesis generation for scientific discovery in the era of large language models. *arXiv preprint arXiv:2504.05496*.
- Azerbaiyev, Z.; Piotrowski, B.; Schoelkopf, H.; Ayers, E. W.; Radev, D.; and Avigad, J. 2023. Proofnet: Autoformalizing and formally proving undergraduate-level mathematics. *arXiv preprint arXiv:2302.12433*.
- Baader, F.; and Nipkow, T. 1998. *Term rewriting and all that*. Cambridge university press.
- Bachmair, L.; Dershowitz, N.; and Plaisted, D. A. 1989. Completion without failure. In *Rewriting Techniques*, 1–30. Elsevier.
- Bachmair, L.; and Ganzinger, H. 1994. Rewrite-based equational theorem proving with selection and simplification. *Journal of Logic and Computation*, 4(3): 217–247.
- Birkhoff, G. 1935. On the structure of abstract algebras. In *Mathematical proceedings of the Cambridge philosophical society*, volume 31, 433–454. Cambridge University Press.
- Bolan, M.; Breitner, J.; Brox, J.; Carlini, N.; Carneiro, M.; van Doorn, F.; Dvorak, M.; Goens, A.; Hill, A.; Husum, H.;

- et al. 2025. The Equational Theories Project: Advancing collaborative mathematical research at scale. *arXiv preprint arXiv:2512.07087*.
- Burris, S.; Sankappanavar, H. P.; et al. 1981. *A course in universal algebra*, volume 78. Springer.
- Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; Pinto, H. P. D. O.; Kaplan, J.; Edwards, H.; Burda, Y.; Joseph, N.; Brockman, G.; et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.
- Chen, Q.; Yang, M.; Qin, L.; Liu, J.; Yan, Z.; Guan, J.; Peng, D.; Ji, Y.; Li, H.; Hu, M.; et al. 2025a. Ai4research: A survey of artificial intelligence for scientific research. *arXiv preprint arXiv:2507.01903*.
- Chen, S.; Chen, Y.; Li, Z.; Jiang, Y.; Wan, Z.; He, Y.; Ran, D.; Gu, T.; Li, H.; Xie, T.; et al. 2025b. Recent advances in large language model benchmarks against data contamination: From static to dynamic evaluation. *arXiv preprint arXiv:2502.17521*.
- Claessen, K.; and Lillieström, A. 2011. Automated inference of finite unsatisfiability. *Journal of Automated Reasoning*, 47(2): 111–132.
- Claessen, K.; and Sörensson, N. 2003. New techniques that improve MACE-style finite model finding. In *Proceedings of the CADE-19 Workshop: Model Computation-Principles, Algorithms, Applications*, 11–27.
- Cobbe, K.; Kosaraju, V.; Bavarian, M.; Chen, M.; Jun, H.; Kaiser, L.; Plappert, M.; Tworek, J.; Hilton, J.; Nakano, R.; et al. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.
- Glazer, E.; Erdil, E.; Besiroglu, T.; Chicharro, D.; Chen, E.; Gunning, A.; Olsson, C. F.; Denain, J.-S.; Ho, A.; Santos, E. d. O.; et al. 2024. Frontiermath: A benchmark for evaluating advanced mathematical reasoning in ai. *arXiv preprint arXiv:2411.04872*.
- Gottweis, J.; Weng, W.-H.; Daryin, A.; Tu, T.; Sirkovic, P.; Myaskovsky, A.; Glowaty, G.; Weissenberger, F.; Orlandi, A.; Popovici, D.; et al. 2026. Accelerating scientific discovery with Co-Scientist. *Nature*, 1–3.
- Hendrycks, D.; Burns, C.; Kadavath, S.; Arora, A.; Basart, S.; Tang, E.; Song, D.; and Steinhardt, J. 2021. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*.
- Hubert, T.; Mehta, R.; Sartran, L.; Horváth, M. Z.; Žužić, G.; Wieser, E.; Huang, A.; Schrittwieser, J.; Schroecker, Y.; Ma-soom, H.; et al. 2026. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 651(8106): 607–613.
- Jain, N.; Gu, A.; Li, W.-D.; Yan, F.; Zhang, T.; Wang, S.; Solar-Lezama, A.; Sen, K.; and Stoica, I. 2025. Live-codebench: Holistic and contamination free evaluation of large language models for code. In *International Conference on Learning Representations*, volume 2025, 58791–58831.
- Janota, M.; Rawson, M.; and Schulz, S. 2026. Case Study: Saturations as Explicit Models in Equational Theories. *arXiv preprint arXiv:2602.16324*.
- Jumper, J.; Evans, R.; Pritzel, A.; Green, T.; Figurnov, M.; Ronneberger, O.; Tunyasuvunakool, K.; Bates, R.; Židek, A.; Potapenko, A.; et al. 2021. Highly accurate protein structure prediction with AlphaFold. *nature*, 596(7873): 583–589.
- Kisielewicz, A. 1997. Austin identities. *algebra universalis*, 38(3): 324–328.
- Knuth, D. E.; and Bendix, P. B. 1970. Simple word problems in universal algebras. In *Computational problems in abstract algebra*, 263–297. Elsevier.
- Kovács, L.; and Voronkov, A. 2013. First-order theorem proving and Vampire. In *International Conference on Computer Aided Verification*, 1–35. Springer.
- Lu, C.; Lu, C.; Lange, R. T.; Foerster, J.; Clune, J.; and Ha, D. 2024. The ai scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*.
- McCune, W. 2003. Mace4 reference manual and guide. *arXiv preprint cs/0310055*.
- Merchant, A.; Batzner, S.; Schoenholz, S. S.; Aykol, M.; Cheon, G.; and Cubuk, E. D. 2023. Scaling deep learning for materials discovery. *Nature*, 624(7990): 80–85.
- Moura, L. d.; and Ullrich, S. 2021. The lean 4 theorem prover and programming language. In *International Conference on Automated Deduction*, 625–635. Springer.
- OpenAI. 2026. An OpenAI Model Has Disproved a Central Conjecture in Discrete Geometry. <https://openai.com/index/model-disproves-discrete-geometry-conjecture/>.
- Rein, D.; Hou, B. L.; Stickland, A. C.; Petty, J.; Pang, R. Y.; Dirani, J.; Michael, J.; and Bowman, S. R. 2023. Gpqa: A graduate-level google-proof q&a benchmark. *arXiv preprint arXiv:2311.12022*.
- Romera-Paredes, B.; Barekatin, M.; Novikov, A.; Balog, M.; Kumar, M. P.; Dupont, E.; Ruiz, F. J.; Ellenberg, J. S.; Wang, P.; Fawzi, O.; et al. 2024. Mathematical discoveries from program search with large language models. *Nature*, 625(7995): 468–475.
- Schulz, S.; Cruanes, S.; and Vukmirović, P. 2019. Faster, higher, stronger: E 2.3. In *International Conference on Automated Deduction*, 495–507. Springer.
- Seely, J.; Imajuku, Y.; Zhao, T.; Cetin, E.; and Jones, L. 2025. Sudoku-Bench: Evaluating creative reasoning with Sudoku variants. *arXiv preprint arXiv:2505.16135*.
- Selman, B.; Mitchell, D. G.; and Levesque, H. J. 1996. Generating hard satisfiability problems. *Artificial intelligence*, 81(1-2): 17–29.
- Smallbone, N. 2021. Twee: an equational theorem prover. In *International Conference on Automated Deduction*, 602–613. Springer.
- Tsoukalas, G.; Lee, J.; Jennings, J.; Xin, J.; Ding, M.; Jennings, M.; Thakur, A.; and Chaudhuri, S. 2024. Putnam-bench: Evaluating neural theorem-provers on the putnam mathematical competition. *Advances in Neural Information Processing Systems*, 37: 11545–11569.
- White, C.; Dooley, S.; Roberts, M.; Pal, A.; Feuer, B.; Jain, S.; Schwartz-Ziv, R.; Jain, N.; Saifullah, K.; Dey, S.; et al. 2025. Livebench: A challenging, contamination-limited llm

benchmark. In *International Conference on Learning Representations*, volume 2025, 91595–91631.

Xin, H.; Guo, D.; Shao, Z.; Ren, Z.; Zhu, Q.; Liu, B.; Ruan, C.; Li, W.; and Liang, X. 2024. Deepseek-prover: Advancing theorem proving in llms through large-scale synthetic data. *arXiv preprint arXiv:2405.14333*.

Xu, C.; Guan, S.; Greene, D.; Kechadi, M.; et al. 2024. Benchmark data contamination of large language models: A survey. *arXiv preprint arXiv:2406.04244*.

Yang, K.; Swope, A.; Gu, A.; Chalamala, R.; Song, P.; Yu, S.; Godil, S.; Prenger, R. J.; and Anandkumar, A. 2023. Leandojo: Theorem proving with retrieval-augmented language models. *Advances in Neural Information Processing Systems*, 36: 21573–21612.

Zheng, K.; Han, J. M.; and Polu, S. 2021. Minif2f: a cross-system benchmark for formal olympiad-level mathematics. *arXiv preprint arXiv:2109.00110*.

Zhu, K.; Chen, J.; Wang, J.; Gong, N.; Yang, D.; and Xie, X. 2024. Dyval: Dynamic evaluation of large language models for reasoning tasks. In *International Conference on Learning Representations*, volume 2024, 18091–18128.